

XOR a string with a Zero

EX NO: 1

DATE:

AIM: To write a C program that contains a string (char pointer) with a value 'Hello World'. The program should XOR each character in this string with 0 and displays the result.

ALGORITHM

- Step 1: Start the program.
- Step 2: Declare and initialize the string as "Hello World".
- Step 3: Find the length of the string.
- Step 4: Traverse each character of the string.
- Step 5: XOR each character with 0.
- Step 6: Display the resulting character.
- Step 7: Stop the program.

// PROGRAM

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str[] = "Hello World";
    char str1[11];
    int i, len;
    len = strlen(str);
    for (i = 0; i < len; i++)
    {
        str1[i] = str[i] ^ 0;
        printf("%c", str1[i]);
    }
    str1[len] = '\0';
    printf("\n");
    return 0;
}
```

Output

Hello World

Result

Thus, the C program to XOR each character of the string with 0 was executed successfully, and the output obtained was: Hello World.

XOR a string with a 127

EX NO: 2

DATE:

AIM: To Write a C program that contains a string (char pointer) with a value 'Hello World'. The program should AND or and XOR each character in this string with 127 and display the result.

ALGORITHM

Step 1: Start.

Step 2: Initialize the string as "Hello World".

Step 3: Display each character after performing AND with 127.

Step 4: Display each character after performing XOR with 127.

Step 5: Stop.

PROGRAM

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str[] = "Hello World";
    int i;
    for (i = 0; i < strlen(str); i++)
        printf("%c", str[i] & 127);
    printf("\n");
    for (i = 0; i < strlen(str); i++)
        printf("%c", str[i] ^ 127);
    return 0;
}
```

Output

Hello World

Result

Thus, the C program to perform **AND (&)** and **XOR (^)** operations with **127** on the string "**Hello World**" was executed successfully, and the output was obtained.

Encryption & Decryption using Cipher Algorithms

EX NO: 3

DATE:

AIM: To Write a Java program to perform encryption and decryption 3 using the following algorithms: a) Ceaser Cipher b) Substitution Cipher c) Hill Cipher.

ALGORITHM

Step 1: Start the program.

Step 2: Enter the plaintext and the key (or key matrix for Hill Cipher).

Step 3: Encrypt the plaintext using the selected cipher algorithm.

Step 4: Decrypt the encrypted text using the same key.

Step 5: Display the encrypted and decrypted text.

Step 6: Stop the program.

// CEASER CIPHER PROGRAM

```
import java.util.Scanner;

public class CaesarCipher {

    // Method for Encryption

    public static String encrypt(String text, int key) {
        StringBuilder result = new StringBuilder();
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
            // Skip spaces
            if (ch == ' ')
                continue;
            if (Character.isUpperCase(ch)) {
                ch = (char) ((ch - 'A' + key) % 26 + 'A');
            } else if (Character.isLowerCase(ch)) {
                ch = (char) ((ch - 'a' + key) % 26 + 'a');
            }

            result.append(ch);
        }
    }
}
```

```

    }
    return result.toString();
}

// Method for Decryption
public static String decrypt(String encrypted, String originalText, int key) {
    StringBuilder result = new StringBuilder();
    int index = 0;
    for (int i = 0; i < originalText.length(); i++) {
        // Restore spaces at original positions
        if (originalText.charAt(i) == ' ') {
            result.append(' ');
        } else {
            char ch = encrypted.charAt(index++);
            if (Character.isUpperCase(ch)) {
                ch = (char) ((ch - 'A' - key + 26) % 26 + 'A');
            } else if (Character.isLowerCase(ch)) {
                ch = (char) ((ch - 'a' - key + 26) % 26 + 'a');
            }
            result.append(ch);
        }
    }
    return result.toString();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter any String: ");
    String input = sc.nextLine();
    System.out.print("Enter the Key: ");
    int key = sc.nextInt();
    String encrypted = encrypt(input, key);
}

```

```

String decrypted = decrypt(encrypted, input, key);
System.out.println("Encrypted String is: " + encrypted);
System.out.println("Decrypted String is: " + decrypted);
sc.close();
}
}

```

OUTPUT

```

Enter any String: HELLO WORLD
Enter the Key: 5
Encrypted String is: MJQQTBTWQI
Decrypted String is: HELLO WORLD

```

// SUBSTITUTION CIPHER PROGRAM

```

import java.util.Scanner;

public class SubstitutionCipher {
    // Original alphabet
    static final String ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    // Substitution alphabet (Key)
    static final String SUBSTITUTE = "QWERTYUIOPASDFGHJKLZXCVBNM";
    // Encryption
    public static String encrypt(String text) {
        StringBuilder result = new StringBuilder();
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
            if (Character.isLetter(ch)) {
                boolean lower = Character.isLowerCase(ch);
                ch = Character.toUpperCase(ch);
                int index = ALPHABET.indexOf(ch);
                char enc = SUBSTITUTE.charAt(index);
                if (lower)
                    enc = Character.toLowerCase(enc);
            }
            result.append(enc);
        }
        return result.toString();
    }
}

```

```

        result.append(enc);
    } else {
        result.append(ch); // Keep spaces, numbers, symbols
    }
}
return result.toString();
}

```

// Decryption

```

public static String decrypt(String text) {
    StringBuilder result = new StringBuilder();
    for (int i = 0; i < text.length(); i++) {
        char ch = text.charAt(i);
        if (Character.isLetter(ch)) {
            boolean lower = Character.isLowerCase(ch);
            ch = Character.toUpperCase(ch);
            int index = SUBSTITUTE.indexOf(ch);
            char dec = ALPHABET.charAt(index);
            if (lower)
                dec = Character.toLowerCase(dec);
            result.append(dec);
        } else {
            result.append(ch);
        }
    }
    return result.toString();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter any String: ");
}

```

```

        String plainText = sc.nextLine();
        String encrypted = encrypt(plainText);
        System.out.println("Encrypted String is: " + encrypted);
        String decrypted = decrypt(encrypted);
        System.out.println("Decrypted String is: " + decrypted);
        sc.close();
    }
}

```

OUTPUT

Enter any String: HELLO WORLD
 Encrypted String is: ITSSG VGKSR
 Decrypted String is: HELLO WORLD

// HILL CIPHER PROGRAM

```

import java.util.Scanner;

public class HillCipher {
    // Function to find determinant
    static int determinant(int key[][]){
        return (key[0][0] * key[1][1] - key[0][1] * key[1][0]);
    }
    // Function to find modular inverse
    static int modInverse(int det) {
        det = det % 26;
        if (det < 0)
            det += 26;
        for (int i = 1; i < 26; i++) {
            if ((det * i) % 26 == 1)

```

```

        return i;
    }
    return -1;
}
// Encryption
static String encrypt(String text, int key[][]){
    text = text.toUpperCase().replaceAll("[^A-Z]", "");
    if (text.length() % 2 != 0)
        text += "X";
    StringBuilder cipher = new StringBuilder();
    for (int i = 0; i < text.length(); i += 2) {
        int a = text.charAt(i) - 'A';
        int b = text.charAt(i + 1) - 'A';
        int c1 = (key[0][0] * a + key[0][1] * b) % 26;
        int c2 = (key[1][0] * a + key[1][1] * b) % 26;
        cipher.append((char) (c1 + 'A'));
        cipher.append((char) (c2 + 'A'));
    }
    return cipher.toString();
}
// Decryption
static String decrypt(String cipher, int key[][]){
    int det = determinant(key);
    int invDet = modInverse(det);
    if (invDet == -1) {
        return "Key matrix is not invertible!";
    }
    int invKey[][] = new int[2][2];
    invKey[0][0] = key[1][1];
    invKey[0][1] = -key[0][1];

```

```

invKey[1][0] = -key[1][0];
invKey[1][1] = key[0][0];
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 2; j++) {
        invKey[i][j] = invKey[i][j] * invDet;
        invKey[i][j] %= 26;
        if (invKey[i][j] < 0)
            invKey[i][j] += 26;
    }
}

StringBuilder plain = new StringBuilder();
for (int i = 0; i < cipher.length(); i += 2) {
    int a = cipher.charAt(i) - 'A';
    int b = cipher.charAt(i + 1) - 'A';
    int p1 = (invKey[0][0] * a + invKey[0][1] * b) % 26;
    int p2 = (invKey[1][0] * a + invKey[1][1] * b) % 26;
    plain.append((char) (p1 + 'A'));
    plain.append((char) (p2 + 'A'));
}

return plain.toString();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int key[][] = new int[2][2];
    System.out.println("Enter 2x2 Key Matrix:");
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)
            key[i][j] = sc.nextInt();
    sc.nextLine();
    System.out.print("Enter Plain Text: ");

```

```
String text = sc.nextLine();  
String cipher = encrypt(text, key);  
System.out.println("Encrypted Text: " + cipher);  
String plain = decrypt(cipher, key);  
System.out.println("Decrypted Text: " + plain);  
sc.close();  
}  
}
```

OUTPUT

Enter 2x2 Key Matrix:

3 3

2 5

Enter Plain Text: HEAT

Encrypted Text: HIFR

Decrypted Text: HEAT

Result

Thus, the Java programs for Caesar Cipher, Substitution Cipher, and Hill Cipher were successfully executed, and the plaintext was encrypted and decrypted correctly.

Java program to implement the DES algorithm logic

EX NO: 4

DATE:

AIM: To write and implement a Java program that demonstrates the logic of the Data Encryption Standard (DES) algorithm for encrypting and decrypting plaintext using a secret key, thereby ensuring secure data transmission through symmetric-key cryptography.

ALGORITHM

- Step 1:** Start the program.
- Step 2:** Read the input string.
- Step 3:** Encrypt the string using the DES secret key.
- Step 4:** Display the encrypted text.
- Step 5:** Decrypt the encrypted text using the same key.
- Step 6:** Display the decrypted text.
- Step 7:** Stop the program.

// DES PROGRAM

```
import java.util.Base64;
import java.util.Scanner;
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;

public class DESAlgorithm {
    public static void main(String[] args) {
        try {
            Scanner sc = new Scanner(System.in);
            // Fixed 8-byte DES key
            String keyString = "12345678";
            SecretKey secretKey = new SecretKeySpec(keyString.getBytes(), "DES");
            // Read input
            System.out.print("Enter the string: ");
            String plainText = sc.nextLine();
```

```

System.out.println("String To Encrypt: " + plainText);
// Encrypt
Cipher encryptCipher = Cipher.getInstance("DES/ECB/PKCS5Padding");
encryptCipher.init(Cipher.ENCRYPT_MODE, secretKey);
byte[] encryptedBytes = encryptCipher.doFinal(plainText.getBytes("UTF-8"));
String encryptedText = Base64.getEncoder().encodeToString(encryptedBytes);
System.out.println("Encrypted Value : " + encryptedText);
// Decrypt
Cipher decryptCipher = Cipher.getInstance("DES/ECB/PKCS5Padding");
decryptCipher.init(Cipher.DECRYPT_MODE, secretKey);
byte[] decryptedBytes = decryptCipher.doFinal(Base64.getDecoder().decode(encryptedText));
String decryptedText = new String(decryptedBytes, "UTF-8");
System.out.println("Decrypted Value : " + decryptedText);
sc.close();
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

OUTPUT

```

Enter the string: RAJAN
String To Encrypt: RAJAN
Encrypted Value : D9esx4jirFQ=
Decrypted Value : RAJAN

```

RESULT

Thus, the Java program implementing the **Data Encryption Standard (DES)** algorithm was successfully executed. The input plaintext was encrypted into the ciphertext using the DES secret key and was successfully decrypted back to the original plaintext , verifying the correctness of the encryption and decryption process.

Java program to implement the Blowfish algorithm logic

EX NO: 5

DATE:

AIM: To implement the **Blowfish algorithm** in Java for encryption and decryption of data using the Java Cryptography Extension (JCE).

ALGORITHM

- Step 1:** Start.
- Step 2:** Input the plaintext.
- Step 3:** Generate a Blowfish secret key.
- Step 4:** Encrypt the plaintext using the Blowfish cipher.
- Step 5:** Decrypt the encrypted text using the same key.
- Step 6:** Display the original, encrypted, and decrypted text.
- Step 7:** Stop.

// BLOWFISH PROGRAM

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Base64;

public class BlowfishExample {
    public static void main(String[] args) {
        try {
            // Plain text
            String plainText = "Hello Blowfish Algorithm";

            // Generate a 128-bit Blowfish key
            KeyGenerator keyGenerator = KeyGenerator.getInstance("Blowfish");
            keyGenerator.init(128);
            SecretKey secretKey = keyGenerator.generateKey();

            // Create Blowfish cipher
            Cipher cipher = Cipher.getInstance("Blowfish");
```

```

// Encrypt the plain text
cipher.init(Cipher.ENCRYPT_MODE, secretKey);
byte[] encryptedBytes = cipher.doFinal(
    plainText.getBytes(StandardCharsets.UTF_8));
String encryptedText = Base64.getEncoder()
    .encodeToString(encryptedBytes);

// Decrypt the encrypted text
cipher.init(Cipher.DECRYPT_MODE, secretKey);
byte[] decryptedBytes = cipher.doFinal(
    Base64.getDecoder().decode(encryptedText));
String decryptedText = new String(
    decryptedBytes, StandardCharsets.UTF_8);

// Display the results
System.out.println("Original Text : " + plainText);
System.out.println("Encrypted Text : " + encryptedText);
System.out.println("Decrypted Text : " + decryptedText);
} catch (Exception e) {
    System.out.println("Error: " + e.getMessage());
}
}
}

```

OUTPUT

Original Text : Hello Blowfish Algorithm

Encrypted Text : bHe1OVe7Wr48XuOJDVUKUPgan+LsCXHnnqxk2xnHyiY=

Decrypted Text : Hello Blowfish Algorithm

RESULT

The Blowfish algorithm was successfully implemented in Java. The plaintext was encrypted using the Blowfish cipher and successfully decrypted back to its original form using the same secret key. Thus, the encryption and decryption process was verified successfully.

Java program to implement the Rijndael algorithm logic

EX NO: 6

DATE:

AIM: To write and execute a Java program to implement the **Rijndael (AES)** algorithm logic for encrypting and decrypting a plaintext message using a secret key.

ALGORITHM

- Step 1:** Start the program.
- Step 2:** Create a secret key for AES (Rijndael).
- Step 3:** Enter the plaintext.
- Step 4:** Encrypt the plaintext using the AES algorithm.
- Step 5:** Display the encrypted text.
- Step 6:** Decrypt the encrypted text using the same key.
- Step 7:** Display the original plaintext.
- Step 8:** Stop the program.

// RIJNDAEL PROGRAM

```
import javax.crypto.Cipher;

import javax.crypto.spec.SecretKeySpec;

import java.nio.charset.StandardCharsets;

import java.util.Base64;

public class RijndaelAES {

    public static void main(String[] args) {

        try {

            // 128-bit (16-byte) AES key

            String key = "1234567890abcdef";

            SecretKeySpec secretKey = new SecretKeySpec(

                key.getBytes(StandardCharsets.UTF_8), "AES");

            // Plain text

            String plainText = "Hello Rijndael AES";

            // Create AES Cipher

            Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
```

```

// Encrypt
cipher.init(Cipher.ENCRYPT_MODE, secretKey);
byte[] encryptedBytes = cipher.doFinal(
    plainText.getBytes(StandardCharsets.UTF_8));
String encryptedText = Base64.getEncoder()
    .encodeToString(encryptedBytes);
System.out.println("Original Text : " + plainText);
System.out.println("Encrypted Text : " + encryptedText);
// Decrypt
cipher.init(Cipher.DECRYPT_MODE, secretKey);
byte[] decryptedBytes = cipher.doFinal(
    Base64.getDecoder().decode(encryptedText));
String decryptedText = new String(
    decryptedBytes, StandardCharsets.UTF_8);
System.out.println("Decrypted Text : " + decryptedText);
} catch (Exception e) {
    System.out.println("Error: " + e.getMessage());
}
}
}

```

OUTPUT

```

Original Text : Hello Rijndael AES
Encrypted Text : KTq3Fe7vnFlibKim5EkJHUmUKWxiKPwd+yibVD/SLg4=
Decrypted Text : Hello Rijndael AES

```

RESULT

Thus, the Java program to implement the **Rijndael (AES)** algorithm was executed successfully. The plaintext was encrypted into ciphertext and then decrypted successfully to obtain the original plaintext.

Using Java Cryptography, encrypt the text “Hello world” using BlowFish. Create your own key using Java keytool

EX NO: 7

DATE:

AIM: To generate a secret key using Java Keytool and implement Blowfish encryption in Java using the Java Cryptography Architecture (JCA) to encrypt the text “Hello world”.

ALGORITHM

- Step 1:** Start the program.
- Step 2:** Generate a Blowfish secret key using **Java Keytool**.
- Step 3:** Load the secret key from the keystore.
- Step 4:** Create a **Blowfish Cipher** using JCA.
- Step 5:** Initialize the cipher for encryption.
- Step 6:** Encrypt “**Hello world**” using doFinal().
- Step 7:** Display the encrypted text in **Base64** format.
- Step 8:** Stop the program.

//BLOWFISH PROGRAM

```
import javax.crypto.Cipher;

import javax.crypto.SecretKey;

import java.util.Base64;

public class BlowfishEncryption {

    public static void main(String[] args) throws Exception {

        // Generate Blowfish secret key

        javax.crypto.KeyGenerator keyGen =

        javax.crypto.KeyGenerator.getInstance("Blowfish");

        SecretKey key = keyGen.generateKey();
```

```
// Create Blowfish cipher

Cipher cipher = Cipher.getInstance("Blowfish");

// Encrypt the text

String text = "Hello world";

cipher.init(Cipher.ENCRYPT_MODE, key);

byte[] encrypted = cipher.doFinal(text.getBytes());

// Display output

System.out.println("Original Text: " + text);

System.out.println("Encrypted Text: " +

    Base64.getEncoder().encodeToString(encrypted));

}

}
```

OUTPUT

Original Text: Hello world

Encrypted Text: Wy9rynTHm8CAWudFhnBfeg==

RESULT

Thus, the Blowfish secret key was generated using Java Keytool, and the text “**Hello world**” was successfully encrypted using Java Cryptography Architecture (JCA).

Write a Java program to implement RSA Algorithm

EX NO: 8

DATE:

AIM: To write and implement a Java program to demonstrate the RSA (Rivest–Shamir–Adleman) public-key cryptographic algorithm for encryption and decryption of messages.

ALGORITHM

- Step 1:** Start.
- Step 2:** Select two prime numbers **p** and **q**.
- Step 3:** Generate RSA public and private keys.
- Step 4:** Encrypt the message using the public key.
- Step 5:** Decrypt the message using the private key.
- Step 6:** Display the encrypted and decrypted messages.
- Step 7:** Stop.

RSA PROGRAM

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.PrivateKey;
import java.security.PublicKey;
import javax.crypto.Cipher;
import java.util.Base64;

public class RSAExample {
    public static void main(String[] args) throws Exception {
        String message = "Hello RSA";
        // Generate RSA key pair
        KeyPairGenerator keyPairGenerator =
            KeyPairGenerator.getInstance("RSA");
        keyPairGenerator.initialize(2048);
        KeyPair keyPair = keyPairGenerator.generateKeyPair();
        PublicKey publicKey = keyPair.getPublic();
        PrivateKey privateKey = keyPair.getPrivate();
        // Encrypt the message using public key
        Cipher cipher = Cipher.getInstance("RSA");
        cipher.init(Cipher.ENCRYPT_MODE, publicKey);
        byte[] encryptedMessage = cipher.doFinal(message.getBytes("UTF-8"));
```

```
// Decrypt the message using private key
cipher.init(Cipher.DECRYPT_MODE, privateKey);
byte[] decryptedMessage = cipher.doFinal(encryptedMessage);
// Display results
System.out.println("Original Message : " + message);
System.out.println("Encrypted Message : "
    + Base64.getEncoder().encodeToString(encryptedMessage));
System.out.println("Decrypted Message : "
    + new String(decryptedMessage, "UTF-8"));
}
```

OUTPUT

Original Message : Hello RSA

Encrypted Message :

edPhDoe0H6bju4QeKCT8SHbY61BiYu4DcGbQCHg/+NeP/DYMUwtRmotoNwvXd3kk7r1ibRefmzANqZhE
XrQzT1VndjcYWB41klj6SBLaUxiws7Z8n6ikGmU8653xzLBe667FB+ZWLe7ApbdYbyPjVEG69Iz93ChVpA
WF0ax6I0HqtCq4ULnGd/iaOCoCuJFukQ5xQeZO35SKdkiUwdC7Yi/ww+VqsROvy43IP4byUYSpjkYCH94z
sq69sRw/rKCTX+JnjINUknWF7ek2a/0MzJ/m0yULOWpiJl2i26dBxVFsfp+MLv4ReNP6gho7UZeYX7QML3f
+wVWE9T/Z1xNA4Q==

Decrypted Message : Hello RSA

RESULT

Thus, the RSA algorithm was successfully implemented in Java, and the message was encrypted using the public key and decrypted using the private key successfully.

Write a Java program to Implement the Diffie-Hellman Key Exchange mechanism

EX NO: 9

DATE:

AIM: To implement the Diffie-Hellman Key Exchange mechanism in Java to generate a shared secret key.

ALGORITHM

- Step 1:** Start.
- Step 2:** Enter prime number **P**, primitive root **G**, and private keys **a** and **b**.
- Step 3:** Calculate public keys: $A = G^a \bmod P$, $B = G^b \bmod P$.
- Step 4:** Calculate shared keys: $KA = B^a \bmod P$, $KB = A^b \bmod P$.
- Step 5:** Compare **KA** and **KB**.
- Step 6:** If equal, display “Key Exchange Successful!”.
- Step 7:** Stop.

// DIFFIE-HELLMAN KEY EXCHANGE PROGRAM

```
import java.util.Scanner;

public class DiffieHellman {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter prime number (P): ");

        long p = sc.nextLong();

        System.out.print("Enter primitive root (G): ");

        long g = sc.nextLong();

        System.out.print("Enter Alice's private key: ");

        long a = sc.nextLong();

        System.out.print("Enter Bob's private key: ");

        long b = sc.nextLong();

        // Public keys

        long A = (long) Math.pow(g, a) % p;
```

```

long B = (long) Math.pow(g, b) % p;
// Shared secret keys
long keyAlice = (long) Math.pow(B, a) % p;
long keyBob = (long) Math.pow(A, b) % p;
System.out.println("\nAlice's Public Key: " + A);
System.out.println("Bob's Public Key: " + B);
System.out.println("Alice's Shared Key: " + keyAlice);
System.out.println("Bob's Shared Key: " + keyBob);
if (keyAlice == keyBob)
    System.out.println("Key Exchange Successful!");
else
    System.out.println("Key Exchange Failed!");
sc.close();
}
}

```

OUTPUT

```

Enter prime number (P): 23
Enter primitive root (G): 5
Enter Alice's private key: 6
Enter Bob's private key: 15
Alice's Public Key: 8
Bob's Public Key: 19
Alice's Shared Key: 2
Bob's Shared Key: 2
Key Exchange Successful!

```

RESULT

Thus, the Diffie-Hellman Key Exchange mechanism was successfully implemented in Java, and both Alice and Bob generated the same shared secret key.

Write a java program for Configuration and Demonstration of SSL/TLS using OpenSSL (Secure Socket Communication)

EX NO: 10

DATE:

AIM: To configure and demonstrate SSL/TLS-based secure socket communication using OpenSSL and Java, ensuring secure and encrypted data transmission between a client and server.

ALGORITHM

- Step 1:** Generate SSL certificate and key using OpenSSL.
- Step 2:** Configure the server keystore and client truststore.
- Step 3:** Create SSL server and client sockets.
- Step 4:** Establish the TLS handshake.
- Step 5:** Exchange encrypted messages.
- Step 6:** Close the connection.

// 1. SSL CLIENT PROGRAM

```
import javax.net.ssl.*;
import java.io.*;
import java.security.KeyStore;

public class SSLClient {
    public static void main(String[] args) {
        try {
            // Load trusted server certificate
            KeyStore trustStore = KeyStore.getInstance("JKS");
            try (FileInputStream fis =
                new FileInputStream("truststore.jks")) {
                trustStore.load(fis, "password".toCharArray());
            }
        }
    }
}
```

```

// Initialize TrustManager
TrustManagerFactory tmf =
    TrustManagerFactory.getInstance(
        TrustManagerFactory.getDefaultAlgorithm());
tmf.init(trustStore);
// Create SSL context
SSLContext context = SSLContext.getInstance("TLS");
context.init(null, tmf.getTrustManagers(), null);
// Create SSL client socket
SSLSocketFactory factory = context.getSocketFactory();
try (SSLSocket socket =
    (SSLSocket) factory.createSocket("localhost", 9999)) {
    // Start TLS handshake
    socket.startHandshake();
    BufferedReader in = new BufferedReader(
        new InputStreamReader(socket.getInputStream()));
    PrintWriter out = new PrintWriter(
        socket.getOutputStream(), true);
    out.println("Hello from SSL Client!");
    System.out.println("Server: " + in.readLine());
}
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

// 2. SSL SERVER PROGRAM

```

import javax.net.ssl.*;
import java.io.*;
import java.security.KeyStore;

public class SSLServer {
    public static void main(String[] args) {
        try {
            // Load server certificate and private key
            KeyStore keyStore = KeyStore.getInstance("PKCS12");
            try (FileInputStream fis = new FileInputStream("server.p12")) {
                keyStore.load(fis, "password".toCharArray());
            }
            // Initialize KeyManager
            KeyManagerFactory kmf = KeyManagerFactory.getInstance(
                KeyManagerFactory.getDefaultAlgorithm());
            kmf.init(keyStore, "password".toCharArray());
            // Create SSL context
            SSLContext context = SSLContext.getInstance("TLS");
            context.init(kmf.getKeyManagers(), null, null);
            // Create SSL server socket
            SSLServerSocketFactory factory =
                context.getServerSocketFactory();
            try (SSLServerSocket serverSocket =
                (SSLServerSocket) factory.createServerSocket(9999)) {
                System.out.println("SSL Server started...");
                System.out.println("Waiting for client...");
                try (SSLSocket socket =
                    (SSLSocket) serverSocket.accept()) {

```

```

        BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
        PrintWriter out = new PrintWriter(
            socket.getOutputStream(), true);
        String message = in.readLine();
        System.out.println("Client: " + message);
        out.println("Hello from SSL Server!");
    }
}
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

3. Generate the required files using OpenSSL

Run these commands in the same folder as the Java programs:

```
openssl req -x509 -newkey rsa:2048 -keyout server.key -out server.crt -days 365 -nodes -subj "/CN=localhost"
```

Create the PKCS12 server keystore:

```
openssl pkcs12 -export -in server.crt -inkey server.key -out server.p12 -name server -passout pass:password
```

Create the client truststore:

```
keytool -importcert -file server.crt -keystore truststore.jks -alias server -storepass password -noprompt
```

4. Compile

```
javac SSLServer.java SSLClient.java
```

5. Run

First run the server:

```
java SSLServer
```

Then open another terminal and run:

```
java SSLClient
```

OUTPUT

Server:

SSL Server started...

Waiting for client...

Client: Hello from SSL Client!

Client:

Server: Hello from SSL Server!

RESULT

SSL/TLS secure socket communication between the Java client and server was successfully established using an OpenSSL-generated certificate, enabling **secure and encrypted data transmission**.